# LEGAL SAHARA

## System/ Functional Design (FD)


**Team Lead:**

Rafsha 26639

**Member(s):**

Hamna Inam Abro 27113

Zara Masood 26928

**PROJECTS COMMITTEE (PC)**

***SUPERVISOR***:

Solat Jabeen Sheikh

IBA SMCS

School of Mathematics and Computer Science

# Table of Contents

# Introduction

## *Overview*

Legal Sahara is an AI-powered legal assistant built using an agentic RAG architecture to support three core functions: semantic case discovery, case summarization, and petition drafting. The system combines document processing, vector-based search, structured data extraction, and large language models to streamline legal research and drafting workflows.

This document outlines Legal Sahara's internal design, including interaction diagrams, class architecture, system components, data storage schema, network protocols, control flow, algorithms, UI considerations, and test cases. It provides a complete technical understanding of how the system operates from user input through AI processing to final output.

# System Design

## Interaction Diagrams (Agent-wise)

### Case Discovery Agent:

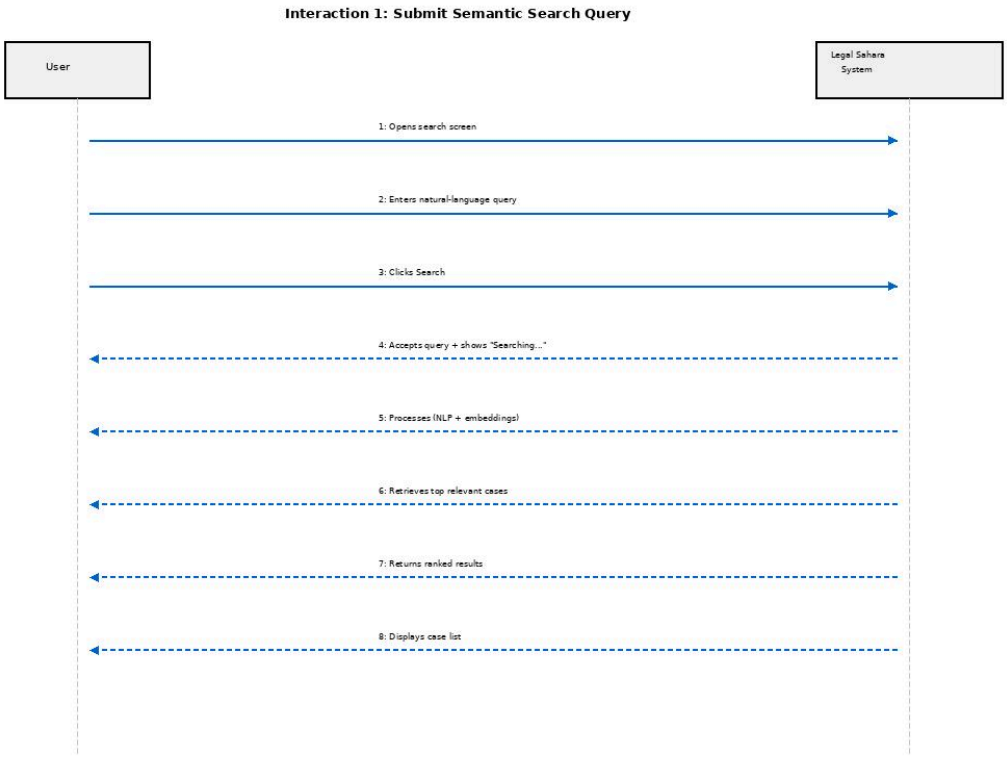



*Figure 1. Submit Semantic Search Query*

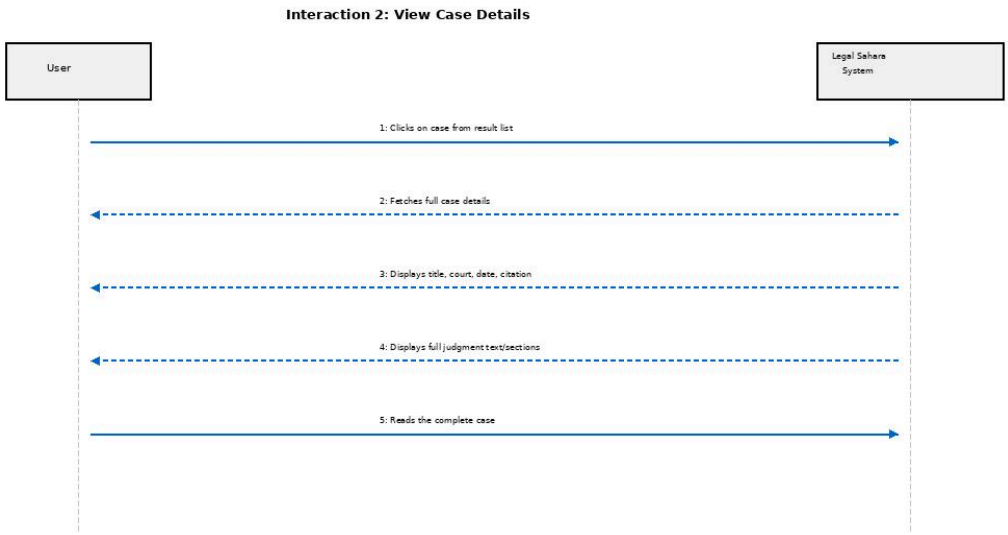



*Figure 2. View Case Details*

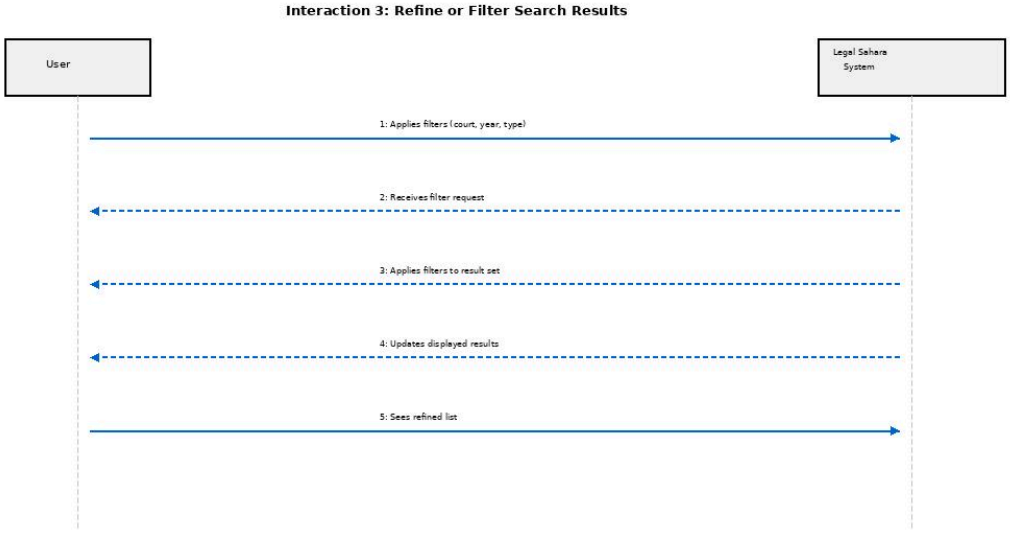



*Figure 3. Refine or Filter Search Result*

## Case Summarizer Agent:


*Figure 4. Summarize an uploaded judgement*

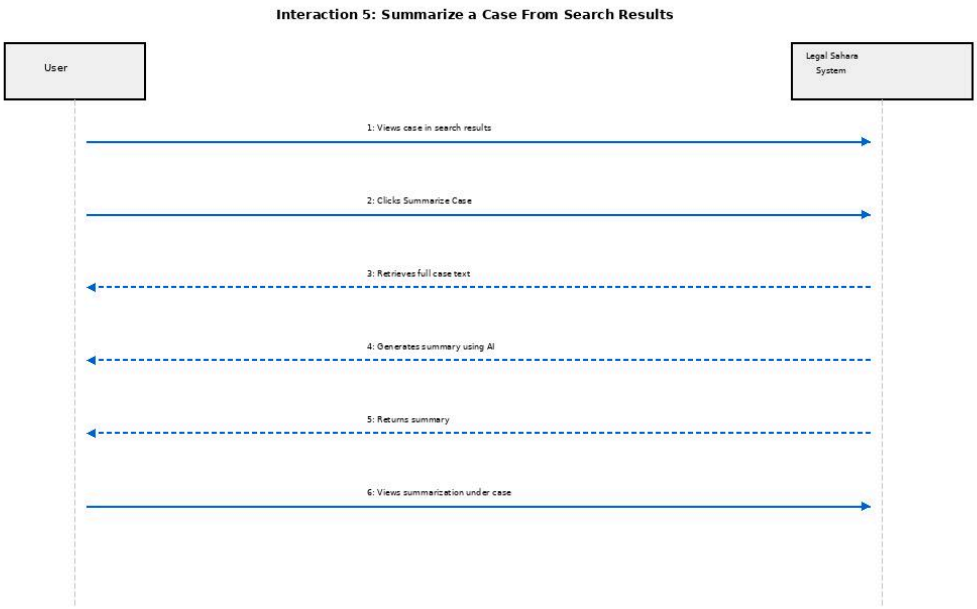



*Figure 5. Summarize Case from Search Results*

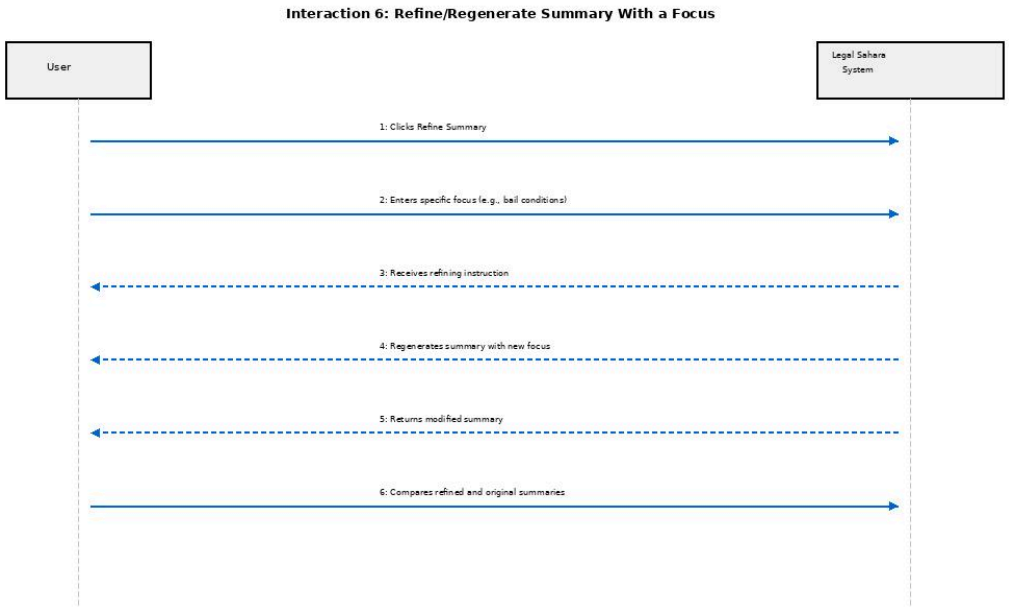



*Figure 6. Refine summary with a focus*

## Petition Drafting Agent:

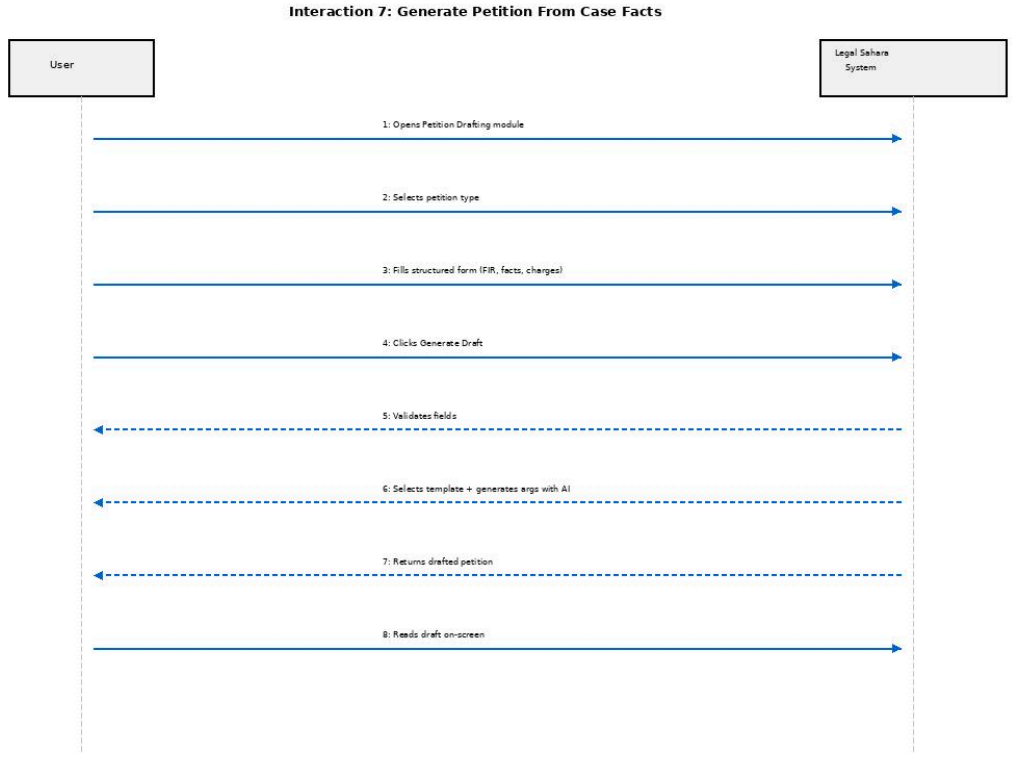



*Figure 7. Generate Petition From Case Facts*

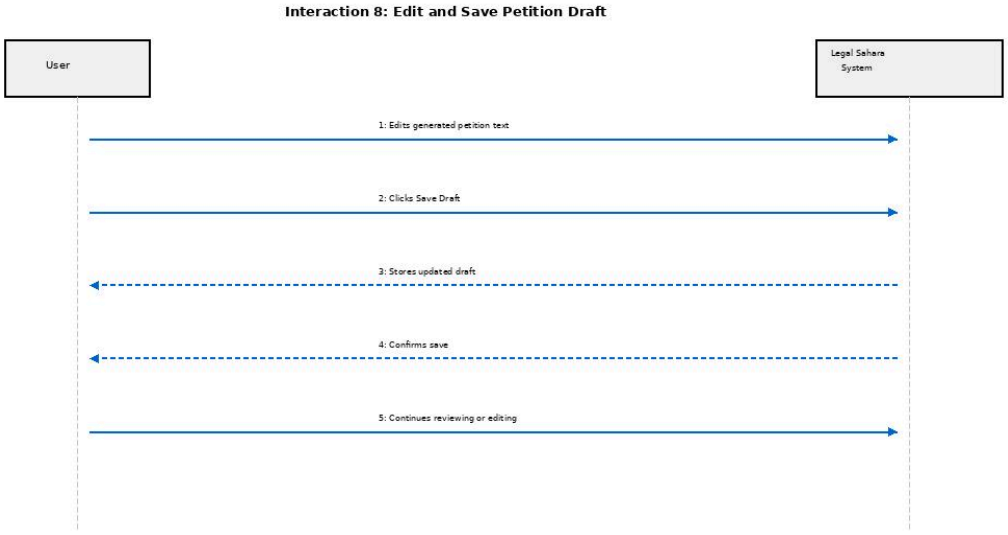



*Figure 8. Edit and save petition draft*

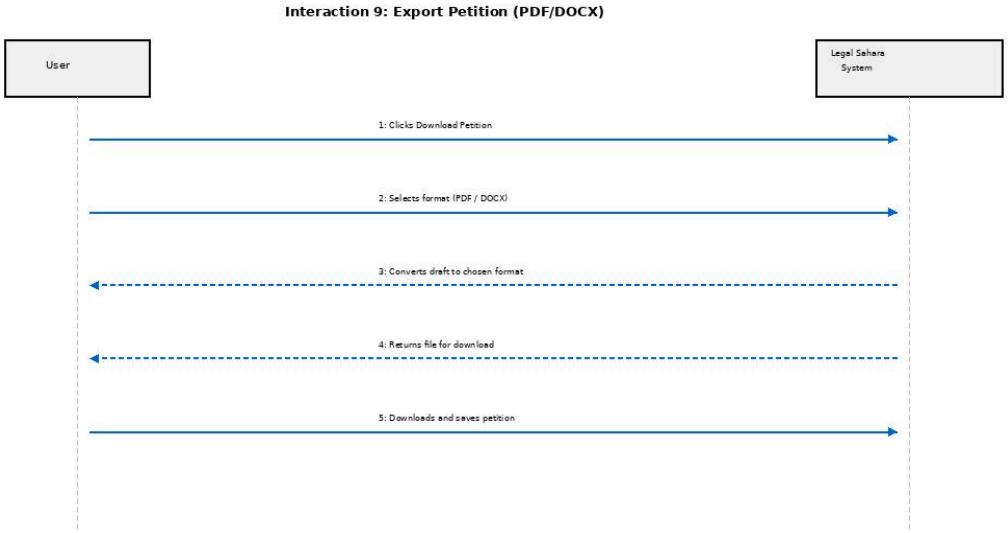
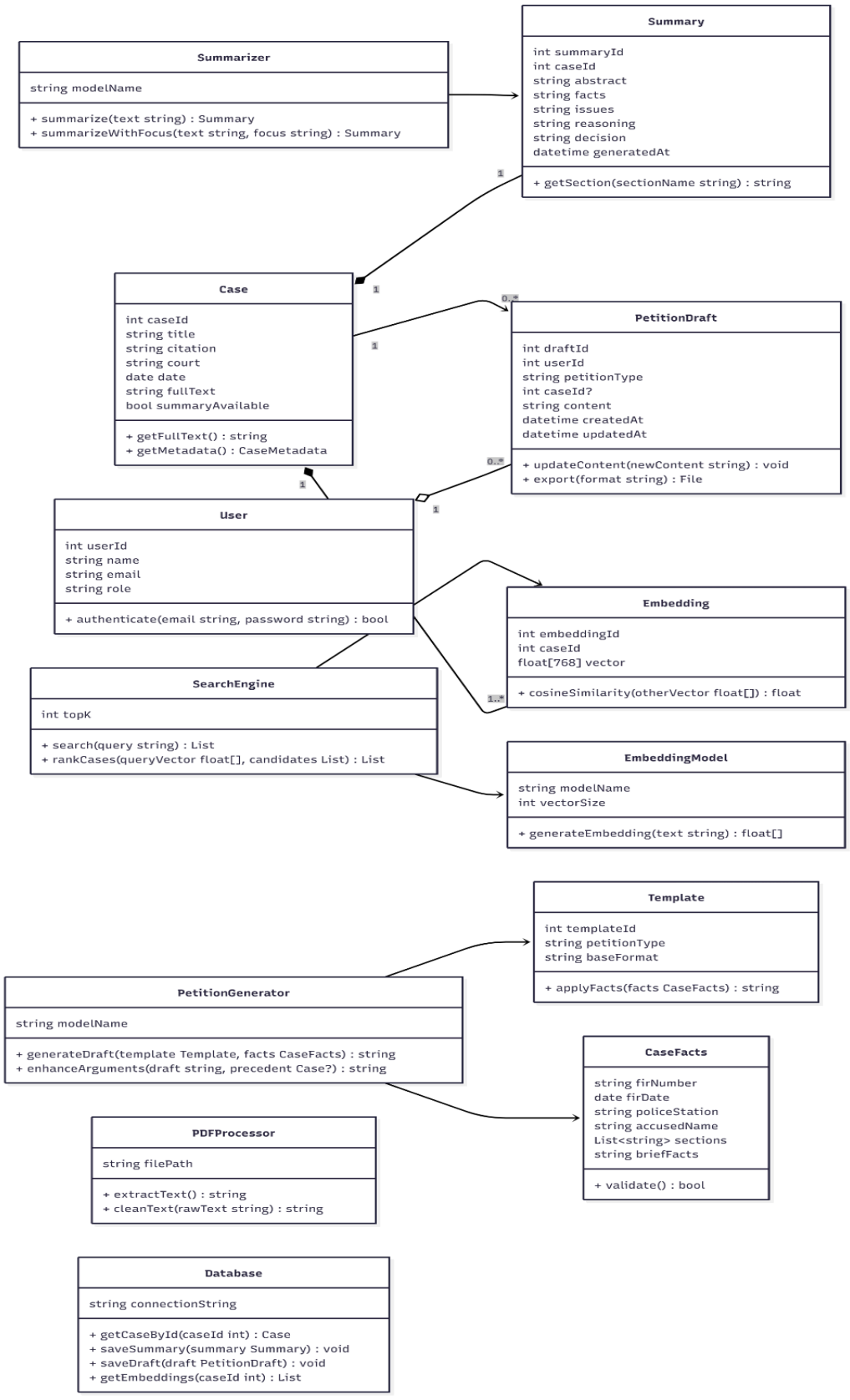



*Figure 9. Export Petition*


*Figure 10. Generate Petition using a selected case*

# Class Diagram and Interface Specification

*Figure 11. Class Diagram - Legal Sahara*



# System Architecture and System Design

## *Architectural Style*

The Legal Sahara system follows a Client–Server Architecture implemented using a Web Frontend–Backend Framework model, optimized for AI-driven legal information processing.

The architecture separates concerns into the following layers:

### 1. Frontend (Client Layer) — React.js Web Application

- Provides the user interface for lawyers, students, and researchers.
- Handles user interactions such as:
    - submitting semantic search queries
    - uploading judgments for summarization
    - entering case facts for petition drafting
- Communicates with the backend via REST APIs over HTTPS.
- Renders search results, summaries, and generated petitions.

### 2. Backend (Server Layer) — FastAPI Service

Acts as the central processing unit of the system:

- Implements REST API endpoints for all core functions
  (search, summarization, petition drafting, file processing).
- Handles NLP pipelines for text cleaning, normalization, and chunking.
- Integrates AI models for embeddings, summarization, and drafting.
- Performs validation (case facts, file formats, user input).
- Orchestrates communication between the Vector Database, relational database, and AI models.
- Ensures secure access control using token-based authentication (JWT/OAuth2 recommended).

### 3. AI Processing Layer (LLM & NLP Pipelines)

Dedicated subsystem responsible for all heavy AI tasks:

- Embedding generation using Sentence-BERT / Instructor / MiniLM.

- Summarization using GPT or custom LLM endpoints.
- Petition drafting using template + LLM-based generation.
- Includes processing modules like:
    - PDF text extraction
    - Text chunking
    - Query vectorization
    - Legal-specific prompt engineering

This layer is abstracted behind service classes, keeping the backend modular.

## 4. Vector Database Layer — FAISS or pgvector

Stores dense semantic embeddings of all legal cases.

- Enables k-Nearest Neighbor (kNN) similarity search.
- Used exclusively by the Case Discovery module.
- Supports indexing, cosine similarity, and ANN algorithms.
- Stores:
    - caseId
    - embedding vectors
    - metadata references

## 5. Relational Database Layer — PostgreSQL

Stores all structured system data, including:

- Case metadata
- Extracted text
- Summaries
- Petition drafts
- User accounts
- Template configurations

Ensures transactional integrity, indexing for fast queries, and relational mapping.

## 6. File Processing Layer — PDF Processor

Handles ingestion of uploaded court judgments:

- Converts PDF → raw text
- Cleans formatting noise
- Handles multi-column and scanned PDFs where possible
- Sends clean text to the summarization pipeline

Operates independently to ensure modularity.

### *Identifying Subsystems*

The Legal Sahara system consists of multiple subsystems, each responsible for a distinct group of functionalities within the AI-powered legal assistant.

#### User Management & Web Portal Subsystem - *(optional)*

- React frontend + auth + roles (lawyer, student, admin).
- Handles all UI, navigation, login, and sends requests to the backend.

#### Case Discovery Agent (RAG-Based Semantic Search Engine)

- Takes a natural-language query from the UI.
- Uses NLP + embedding model to encode the query.
- Talks to the Vector Store + Case Store to retrieve and rank cases.
- Returns a ranked list of precedents to the UI.

#### Case Summary Agent (Case Summary Generator)

- Takes either an uploaded PDF or a stored case.
- Uses PDF processor + LLM to generate a structured summary (facts, issues, reasoning, decision, timeline).
- Stores summaries for later reuse.

#### Petition Drafting Agent (Petition Generation Agent)

- Takes structured case facts (FIR, sections, accused details) and optional precedents.
- Applies a template and calls an LLM to generate a full petition draft.
- Supports editing, saving, exporting.

## Shared Data & AI Services Subsystem

- Vector DB (FAISS/pgvector) for embeddings.
- Relational DB (PostgreSQL) for structured data (cases, summaries, drafts, users).
- PDF storage / file system.
- EmbeddingModel, LLM client, PDFProcessor – reusable across agents.

## Document & Data Management Subsystem

- Manages ingestion and storage of legal documents (judgment PDFs, extracted text).
- Maintains relational data such as case metadata, summaries, petition drafts, and user information in PostgreSQL.
- Maintains high-dimensional case embeddings in a vector store (FAISS/pgvector) for semantic search.
- Exposes repository interfaces to the search, summarization, and drafting subsystems.

## Web Portal (Frontend) Subsystem

- Implements the React.js web interface used by all users.
- Provides screens for semantic search, case viewing, summarization, and petition drafting.
- Communicates with the backend via REST APIs over HTTPS.
- Handles client-side validation, navigation, and presentation logic.

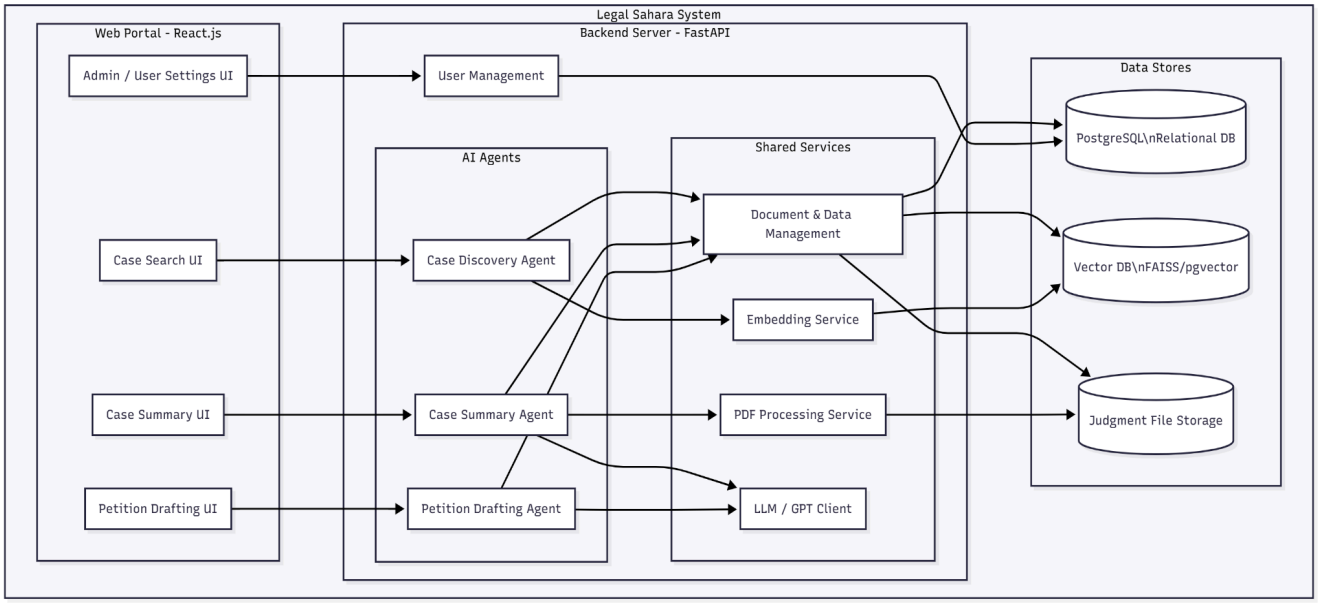



*Figure 11. Package UML*

## *Mapping Subsystems to Hardware*

Since Legal Sahara is an agentic, RAG-based client–server system, the deployment is distributed across multiple machines and services, as shown in the table below.

The three core agents (Case Discovery Agent, Case Summary Agent, Petition Drafting Agent) all run inside the backend application server and rely on shared storage and external/model services.

| Subsystem / Component | Runs On |
|---|---|
| Web Portal (React.js Frontend) | User browsers on client machines (laptops/desktops in chambers/offices) |
| Backend Server (FastAPI + AI Agents) | Cloud Application Server (AWS / Azure / GCP VM or container cluster) |
| – Case Discovery Agent (RAG Semantic Search Engine) | Cloud Application Server (same backend instance) |
| – Case Summary Agent (Summarization Agent) | Cloud Application Server (same backend instance) |
| – Petition Drafting Agent (Petition Generation Agent) | Cloud Application Server (same backend instance) |
| Embedding & RAG Services (EmbeddingModel, SearchEngine) | Cloud Application Server with CPU/GPU support (co-located with backend) |
| Relational Database (PostgreSQL) | Managed Cloud Database Server (e.g., AWS RDS, Azure Database for PG) |
| Vector Database (FAISS / pgvector) | Cloud Database Server (same PostgreSQL instance with pgvector, or a separate vector store VM) |

| | |
|---|---|
| Judgment File Storage (PDF Repository) | Cloud Object Storage (e.g., AWS S3, Azure Blob) or attached storage to backend server |
| External LLM Provider (GPT-based APIs) | Managed LLM service over the internet (OpenAI / Azure OpenAI / similar) |
| Admin & Monitoring Tools (logs, dashboards) | Cloud monitoring/observability services (e.g., CloudWatch, Grafana) |

*Table 1. Mapping Subsystems to Hardware*

## *Persistent Data Storage*

Legal Sahara uses a combination of:

1. **File Storage** – for raw judgment PDFs and generated documents.
2. **Structured JSON** – parsed representation of each judgment.


```
{
  "doc_id": "SC-PK_C.A.11_2022",
  "source_pdf": "C.A.11_2022.pdf",
  "court": "Supreme Court of Pakistan",
  "jurisdiction": "Civil",
  "case_numbers": [],
  "title": "Mr. Sajjad Haider, Capital T.V.",
  "bench": [
    "MR. JUSTICE IJAZ UL AHSAN",
    "MR. JUSTICE MUNIB AKHTAR",
    "MR. JUSTICE SAYYED MAZAHAR ALI AKBAR NAQVI",
    "Civil Appeal No.11 of 2022",
    "(On appeal against"
  ],
  "dates": {
    "hearing": "10.11.2022"
  },
  "statutes_cited": [
    {
      "name": "This appeal arises in relation to the Pakistan \nElectronic Media Regulatory Authority Ordinance, 2002",
      "sections": [],
      "articles": []
```


*Figure 12. Parsed JSON Schema*

3. **Relational Database (PostgreSQL)** – for all structured metadata, summaries, drafts, users, templates, logs.

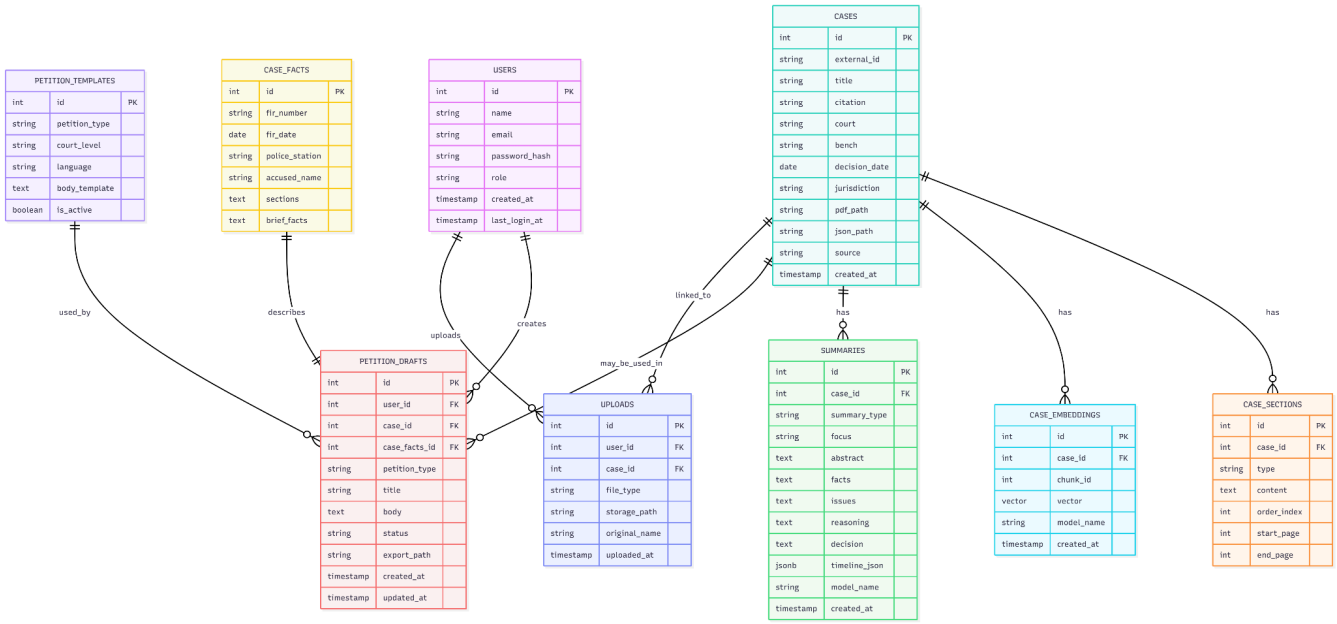



*Figure 13. Relational Database Schema*

4. **Vector Store (pgvector / FAISS)** – for dense embeddings used in the RAG pipeline

## Network Protocol

Since Legal Sahara is a web-based, agentic RAG system deployed in a client–server environment, all major components communicate over standard, secure network protocols. The table below summarizes the protocols used by different features of the system:

| Feature / Channel | Protocol / Technology |
|---|---|
| Frontend ↔ Backend API Communication | REST over HTTPS (HTTP/1.1 or HTTP/2) |
| AI Agent Calls to External LLM APIs | HTTPS (JSON over HTTP to LLM provider) |
| Frontend Asset Delivery (React App) | HTTPS (served via CDN / web server) |
| Database Communication (PostgreSQL) | PostgreSQL native protocol over TLS |

| | |
|---|---|
| Vector DB (pgvector / FAISS Service) | TCP over LAN / HTTPS (if exposed as service) |
| File Storage Access (PDF / JSON store) | HTTPS (to object storage, e.g., S3) or NFS over LAN |

*Table 2. Network Protocols*

## Global Control Flow

The system follows a hybrid control flow:

### Linear Execution:

Most user-facing operations follow a synchronous, request–response pattern over REST:

- User Authentication & Authorization
  - Login, token validation, role checks.
  - Handled via standard REST calls over HTTPS.

- Case Discovery (Semantic Search)
  - User submits a query → Backend (Case Discovery Agent) processes → Returns ranked cases.
  - The full RAG pipeline (query embedding → vector search → metadata fetch) completes within a single HTTP request.

- Case Summarization (On-Demand)
  - User uploads a PDF or selects a stored case → Backend (Case Summary Agent) extracts text and calls the LLM → Returns summary once ready.
  - If the model response is fast enough, this remains a synchronous call.

- Petition Drafting
  - User submits structured case facts → Petition Drafting Agent applies template + LLM → Returns draft petition within the same request.

In all of these linear flows, the user waits for a single response, and operations are executed in a clear, sequential manner inside the backend.

### Event-Driven / Asynchronous Execution:

Certain internal tasks are handled in an event-driven or asynchronous fashion to keep the UI responsive and to offload heavy work:

- Background Embedding Generation & Reindexing
    - When new judgments are added or existing ones are updated, a background job generates embeddings and updates the vector index.
    - These jobs are triggered by events (e.g., “new case added”) rather than by direct user interaction.

- Deferred Summarization / Caching
    - For popular cases, summaries may be pre-generated or refreshed asynchronously and stored in the database.
    - A case view or search “Summarize” button can either serve an existing summary or trigger a background summarization process.

- Periodic Maintenance Tasks
    - Cleanup of stale drafts, temporary files, and old tokens can be scheduled and run independently of user actions.

### Time Dependency:

The system uses time-based behavior mainly in the following ways:

- Request Timeouts

    - Calls to external LLM providers and internal heavy tasks are bound by timeout limits to prevent indefinitely hanging requests.
    - If a timeout occurs, the system returns a controlled error message to the user.

- Scheduled Jobs (Cron / Scheduler)
    - Periodic tasks such as:
        - re-indexing or refreshing embeddings for updated judgments,
        - database backups,
        - cleanup of expired authentication tokens,
        - optional batch summarization of newly ingested cases.

    - These are scheduled using server-side schedulers or job queues (e.g., Celery/Redis, cron-like schedulers)

There is no hard real-time requirement (like GPS updates), but there is an emphasis on **bounded latency** for search, summarization, and drafting operations.

### Concurrency Management:

Legal Sahara is designed to handle **multiple users and requests concurrently**:

- Async Request Handling
    - The backend (FastAPI) uses asynchronous request handling and is typically deployed behind a process manager (e.g., Gunicorn/Uvicorn) with multiple workers.
    - Each incoming HTTP request (search, summarize, draft) can be processed in parallel by different workers, depending on server resources.

- Database Transactions
    - PostgreSQL is used with **transactional integrity** to ensure consistency when:
        - saving petition drafts,
        - updating summaries,
        - writing embeddings,
        - linking cases to uploads.
    - This prevents race conditions such as partially written drafts or duplicate records.

- Concurrent Draft Editing Safeguards
    - When a user edits a petition draft, updates are applied with versioning or “last updated” checks to avoid overwriting changes if multiple edits occur in quick succession.

## Hardware/Sofware Requirements

The following table lists the minimum hardware and software requirements for deploying and running the Legal Sahara (Agentic RAG Legal Assistant) system.

| Component | Minimum Requirement |
|---|---|
| *Client-Side (Web Portal)* | |
| Hardware | Any modern laptop/desktop with 4 GB RAM, dual-core CPU, 1366×768 resolution |
| Software | Modern web browser (Chrome/Firefox/Edge) with JavaScript enabled, stable internet |
| *Server-Side – Backend (FastAPI + Agents)* | |
| Hardware | 4 vCPUs, 8 GB RAM, 80–100 GB SSD storage (for code, cache, logs) |
| Software | Ubuntu 20.04+ LTS or equivalent Linux, Python 3.10+, FastAPI, Uvicorn/Gunicorn, Nginx/Apache reverse proxy |
| *Database & Vector Store* | |
| Hardware | 2–4 vCPUs, 8 GB RAM, 100+ GB SSD (depending on number and size of cases) |
| Software | PostgreSQL 14+ with pgvector extension **or** PostgreSQL + external FAISS service |

| | |
|---|---|
| *Storage (Judgment PDFs & JSONs)* | |
| Hardware | Cloud object storage bucket or file server with scalable disk (start at 100 GB) |
| Software | AWS S3 / Azure Blob / MinIO or equivalent, HTTPS-enabled access |
| *AI / LLM Integration* | |
| Hardware | If using external LLM (recommended): no GPU required on-prem; stable internet. If self-hosting: 1+ GPU with 16 GB VRAM minimum. |
| Software | Access to LLM provider APIs (e.g., OpenAI/Azure), Python SDK/HTTP client libraries |
| *Development & Tooling (Optional)* | |
| Hardware | Developer machine with 8 GB RAM, dual/quad-core CPU |
| Software | Node.js 18+ (for React build), npm/yarn, Git, Docker (optional), VS Code or similar |

*Table 3. Hardware & Software Requirements*

# Algorithms Used

## 1. Semantic Case Discovery (RAG-Based Search Algorithm)

**Purpose**:
To find the most relevant judgments for a user's natural-language legal query using semantic similarity instead of keyword matching.

**Algorithm Type:**
Dense Vector Retrieval with Cosine Similarity (k-Nearest Neighbors)

**Steps:**

1. Receive Query
    - User enters a free-text query (e.g., "post-arrest bail in narcotics case for first-time offender").
    - The query is sent to the Case Discovery Agent.

2. Preprocess Query
    - Normalize text (lowercasing, removing extra whitespace).
    - Optionally detect key sections like PPC/CrPC sections using simple regex patterns.

3. Generate Query Embedding
    - Pass the cleaned query text to the EmbeddingModel.
    - Obtain a dense vector q ∈ R^d (e.g., d = 768).

4. Vector Search in Embedding Store
    - Use the vector database (case_embeddings + pgvector / FAISS) to find top-k most similar case vectors:
        - Compute cosine similarity between q and each case embedding vector.
        - Or use Approximate Nearest Neighbor (ANN) index if dataset is large.

5. Retrieve Candidate Cases
    - Get top-k embedding matches -> case IDs.
    - Query the cases table to fetch titles, citations, courts, decision dates, etc.

6. Apply Metadata Filters (Optional)
    - If the user specified filters (court, year range, jurisdiction), filter candidate cases accordingly.

7. Rank and Return Results
    - Order the final set primarily by similarity score and secondarily by recency or court hierarchy.
    - Return the ranked list to the frontend.

## 2. Case Summarization Algorithm

**Purpose**:
To convert long legal judgments (PDFs) into concise, structured summaries for quick understanding.

**Algorithm Type:**
Two-stage pipeline: Document Processing + LLM-Based Summarization

**Steps:**

1. Input Source
    - Either: user uploads a PDF, or selects an existing case from the system.

2. PDF to Text Extraction
    - The PDFProcessor reads the PDF from storage.
    - Extracts raw text (pages, paragraphs).
    - Removes noise such as headers/footers, page numbers.

3. Text Cleaning and Segmentation
    - Normalize whitespace, remove weird characters.
    - Optionally segment text into logical parts (facts, issues, arguments, decision) using keyword heuristics or headings.

4. Chunking (if needed)
    - For very long judgments, split text into manageable chunks while preserving context.
    - Each chunk is fed separately to the summarization model.

5. LLM Summarization
    - Pass the cleaned/chunked text to the Summarizer (LLM/GPT client) with a prompt such as:
        - “Summarize this judgment into: Facts, Issues, Reasoning, Decision.”

    - For chunked inputs, generate partial summaries and then merge them into a single coherent summary.

6. Structured Output Assembly
    - Combine abstract, key facts, issues, reasoning, and decision into a structured object.
    - Optionally construct a timeline of events and store as JSON.

7. Store and Return
    - Save the summary in the summaries table, linked to cases.id.
    - Return the summary to the frontend for display.

## 3. Petition Drafting Algorithm

**Purpose**:
To generate a first draft of a legally formatted petition (e.g., bail application) based on case facts and templates.

**Algorithm Type**:
Template Filling + LLM-Based Generation

**Steps**:

1. Collect Case Facts
    - User fills a structured form: FIR number/date, sections, police station, accused name, brief facts.
    - Data is stored in case_facts.

2. Select Petition Template
    - Based on petition_type, court level, and language, select a template from petition_templates.
    - Template contains placeholders like {{accusedName}}, {{FIRNumber}}, {{Sections}}.

3. Apply Template with Facts
    - The PetitionGenerator replaces placeholders with actual values from case_facts.
    - Produces a draft skeleton with headings, parties, and basic structure filled.

4. Enhance with LLM
    - Send the structured facts + skeleton draft to the LLM with a prompt like:

        - "Expand and refine this draft into a formal bail application in legal language, maintaining structure and adding legal reasoning."

    - The LLM returns a fully written petition body.

5. Post-Processing
    - Ensure correct formatting (paragraph breaks, numbering).
    - Optionally insert references to precedents chosen by the user.

6. Store & Present
    - Save the draft in petition_drafts.
    - Return it to the frontend editor where user can review, edit, and export.

## 4. PDF Ingestion & Case Creation Algorithm

**Purpose**:
To onboard new judgments into the system and prepare them for search and summarization.

**Steps**:

1. User or admin uploads a PDF.
2. System stores the file and records it in `uploads`.
3. PDFProcessor extracts text and basic metadata (title, court, date if present).
4. Create a record in `cases` with metadata and file paths (PDF and JSON).
5. Optionally parse sections and fill `case_sections`.
6. Trigger background job(s) for:
    - Embedding generation (for `case_embeddings`)
    - Pre-summarization (for `summaries`, optional)

This feeds the RAG pipeline with fresh cases.

### 5. Ranking & Filtering Algorithm

**Purpose**:
To refine search results based on court level, year range, and relevance.

**Steps**:

1. Start with top-k results from semantic search.
2. Filter out cases that do not match user-specified metadata constraints (e.g., court, date).
3. Apply a combined scoring function, for example:
4. Sort cases by final score in descending order.
5. Return top N results.

## Data Structures Used

### 1. Vectors / Arrays

- Embedding vectors are represented as fixed-length float arrays (e.g., float[768]).

These arrays form the core data structure of the RAG engine**.**

### 2. Lists / Arrays (Dynamic)

- In code, typically implemented as List<Case>, List<Embedding>, List<string>.

### 3. Hash Maps / Dictionaries

- Backend often uses hash maps (e.g., Python dictionaries) for:
    - Mapping case_id → metadata in memory caches.
    - Fast lookup of templates by petition_type.
    - Caching summary results by case_id.
- Provides O(1) average lookup time for frequent access patterns.

### 4. JSON / JSONB Structures

- Many parts of the system use JSON as a flexible data structure:

- Parsed judgment JSON (sections, parties, citations).
- summaries.timeline_json for event timelines.
- sections in case_facts when structured.

- In PostgreSQL, jsonb allows indexing and querying nested fields efficiently.

## User Interface Design and Implementation

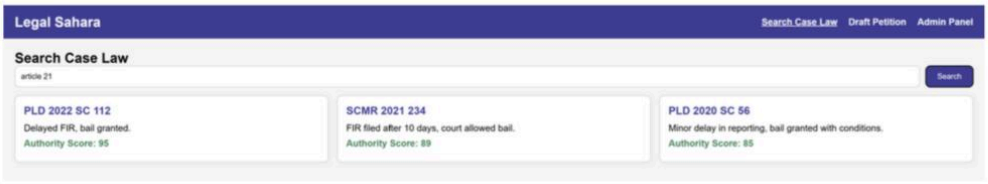



*Figure 14. Search Engine Screen*

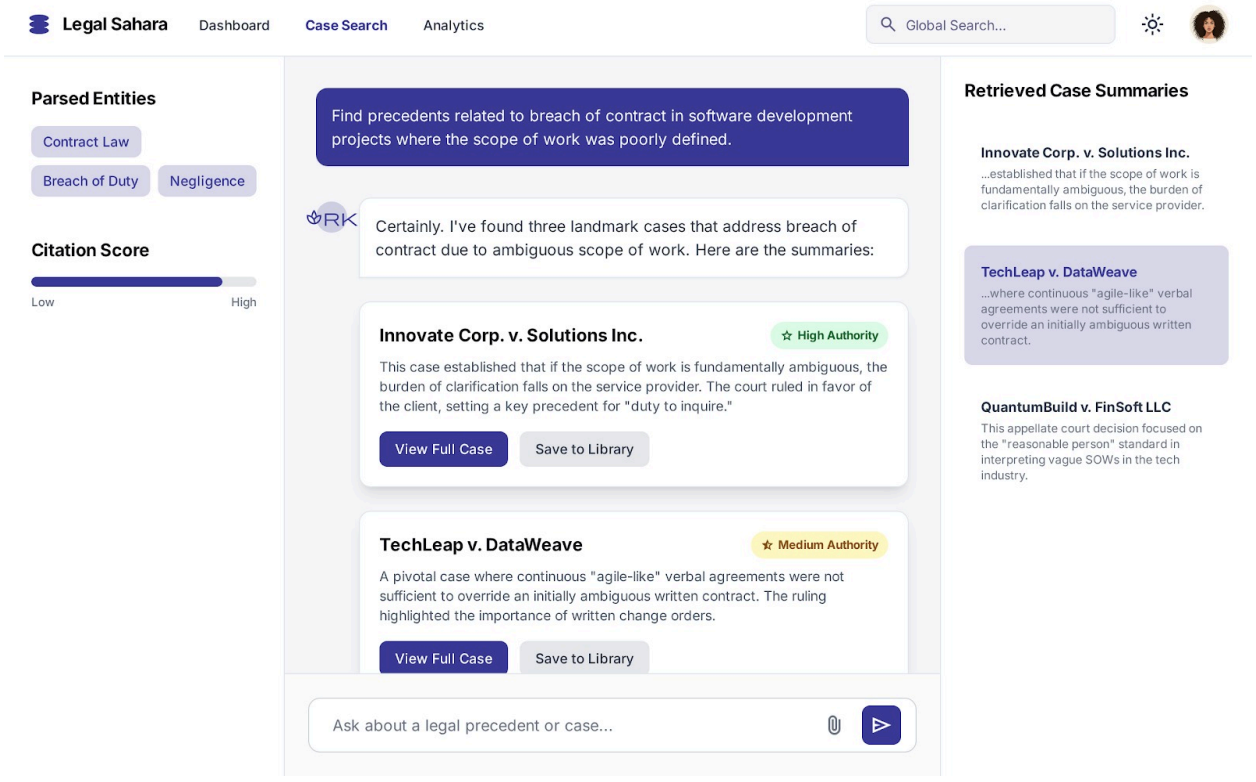



*Figure 15. Search Query*

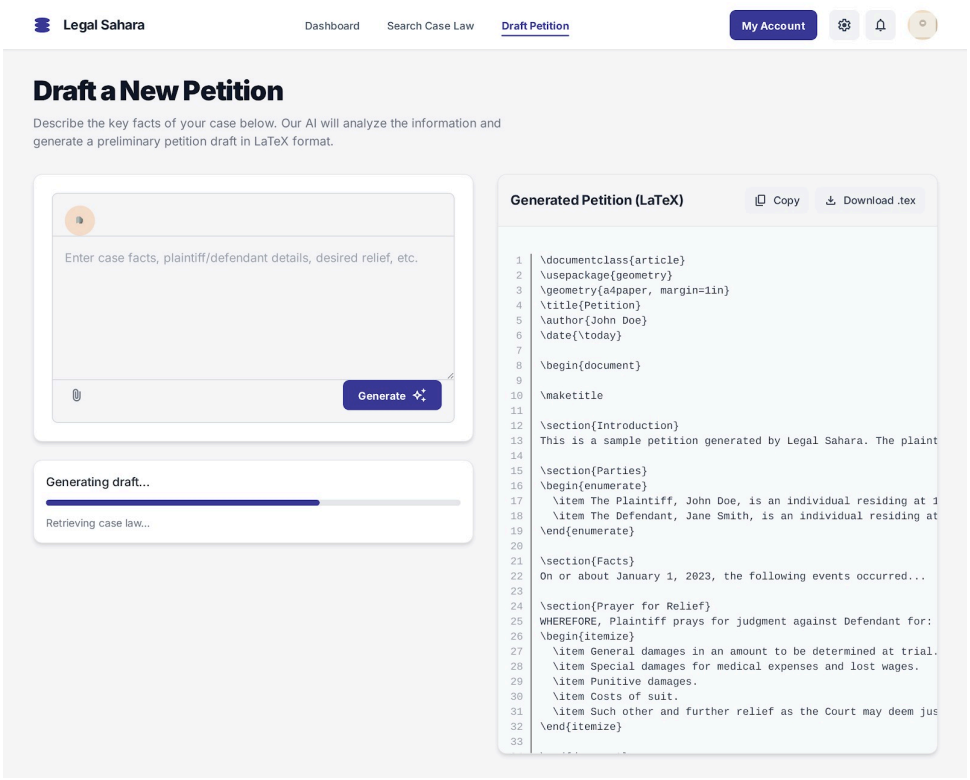



*Figure 16. Petition Drafter*

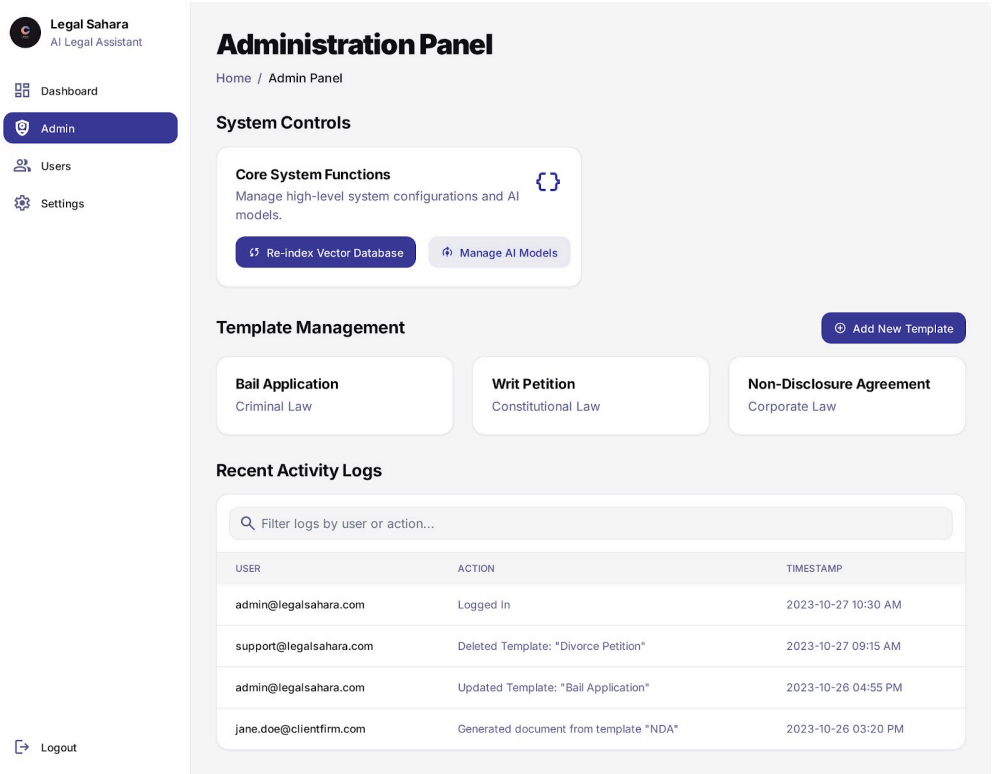



*Figure 17. Admin Panel*

## Design of Tests

| S.No | Test Case | Description | Expected Output |
|---|---|---|---|
| 1 | User Login Authentication | Verify that a user can log in with valid credentials and is rejected with invalid ones. | On valid credentials: JWT/token is issued and user session starts. On invalid: error message returned. |
| 2 | Submit Semantic Search Query | Test SearchEngine.search() with a simple legal query. | A non-empty, ranked list of Case objects is returned without errors. |
| 3 | Semantic Relevance for Similar Queries | For two semantically similar queries, ensure overlapping top results from the Case Discovery Agent. | The top-k result sets have a significant overlap (same key cases appear in both result lists). |
| 4 | Apply Search Filters (Court/Year) | Ensure that filters (court, year range) are correctly applied after semantic retrieval. | All returned cases match the filter criteria (e.g., only Supreme Court, only cases within given years). |
| 5 | View Case Details by ID | Test retrieval of a case by ID from the cases table and related sections. | Full case metadata and associated sections are loaded correctly; missing ID returns a not-found error. |

| | | | |
|---|---|---|---|
| 6 | Upload Judgment PDF & Store Metadata | Verify upload handling and creation of a cases + uploads record. | PDF is saved to storage; cases row and uploads row are created with correct paths and metadata. |
| 7 | Extract Text from PDF | Unit test PDFProcessor.extractText() on a sample judgment PDF. | Returned text is non-empty, readable, and contains expected key phrases from the document. |
| 8 | Generate Summary from Raw Text | Test Summarizer.summarize() with a known input text. | A Summary object is returned with non-empty fields (abstract, facts, issues, reasoning, decision). |
| 9 | Summarize Stored Case by ID | Ensure the Case Summary Agent can fetch case text and create a summary for a given case_id. | Summary is created, stored in summaries, and linked correctly to the given case_id. |
| 10 | Focused Summary Generation | Test summarizeWithFocus(text, focus) with a specific focus (e.g., “bail grounds”). | Output summary emphasizes requested focus terms; focus field is stored correctly in summaries.focus. |
| 11 | Generate Petition from Case Facts | Verify PetitionGenerator.generateDraft(template, facts) with valid CaseFacts and template. | A legally structured petition string is generated, containing all |

| | | | |
|---|---|---|---|
| | | | provided facts in the correct places. |
| 12 | Validate Case Facts Input | Unit test CaseFacts.validate() with missing required fields (e.g., no FIR number). | Validation returns false or throws a validation error; backend responds with a clear error message. |
| 13 | Save Petition Draft | Test saving a newly generated petition into petition_drafts. | A new row appears in petition_drafts with correct user_id, case_facts_id, and body contents. |
| 14 | Export Petition as DOCX/PDF | Verify that an existing petition draft can be exported to DOCX/PDF. | A file is generated, stored at export_path, and a valid download link or file is returned to the user. |
| 15 | Embedding Generation for Case Text | Test EmbeddingModel.generateEmbedding(text) for a sample case chunk. | A fixed-length float vector is returned (e.g., length 768) with no NaNs; stored correctly in DB/store. |
| 16 | Embedding Lookup & Vector Search | Ensure case_embeddings can be queried for nearest neighbors using the vector index. | Query returns the correct nearest case IDs for a known test query vector. |

| | | | |
|---|---|---|---|
| 17 | External LLM Timeout & Error Handling | Simulate LLM API timeout or error during summarization/drafting. | Backend returns a graceful error response to the user; no partial/invalid data is stored in the database. |

*Table 4. Design of Tests*

# Appendix:

List Of Images



List Of Tables